RESEARCH PROPOSAL

ChartAgent

Atomic-Grain Programmatic Data Synthesis for Chart Understanding Benchmarks

Operator Algebra | **Agentic Data Simulator** | **Table Amortization**

One Master Table → 10–30+ Coherent Multi-Chart Tasks

Deterministic · Type-Safe · Cross-Chart Consistent by Construction

Why existing chart QA benchmarks fall short

Three structural deficits shared by ChartQA, PlotQA, ChartBench

01

Shallow data depth

Public datasets store only the 5-row aggregate shown in the chart.

region	revenue
North	$4.2M
South	$3.1M
East	$2.8M

only aggregates — no atomic events

No drill-down, no provenance, no long-tail statistics.

02

Arithmetic hallucinations

LLM-written JSON introduces numbers that don't add up.

10 + 15 = 30

LLM-produced numbers silently break arithmetic

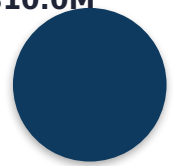
Validation catches syntax — not semantic impossibility.

03

No cross-chart consistency

A pie and a bar from the same domain disagree on totals.

Pie → $10.0M



Bar → $11.4M

same data · totals disagree

Multi-chart reasoning becomes impossible to evaluate.

Comparison with prior work

Every prior benchmark fails ≥ 4 of the 7 axes we care about

Dimension	ChartQA	PlotQA	ChartBench	Ours
Data source	Web-crawled	Templates	Hybrid	**LLM-authored SDK programs**
Data depth	Aggregates	Aggregates	Aggregates	**Atomic-grain fact tables**
Distribution control	None	Uniform/Normal	Semi-fixed	**Mixtures / copulas / conditional**
Cross-chart consistency	✗	✗	✗	✓ **(shared Master Table)**
Error correction	Manual	None	Heuristic	**Code-execution feedback loop**
Multi-chart reasoning	None	None	Limited	**Dashboard-level cross-chart QA**
Reproducibility	✗	✓	Partial	✓ **(SDK + seed)**

Key difference: we control data depth, distribution, and cross-chart semantics — not just the surface image.

Three interlocking contributions

A formal substrate → a programmable simulator → a deterministic amortizer

1 FORMAL

Operator Algebra

Every question is a typed pipeline over Set / Scalar / Bridge operators.

- 16 operators, 2 types (V, S)
- Difficulty = # ops
- Composable multi-hop reasoning

2 EXECUTABLE

Agentic Data Simulator

LLM writes Python calling a type-safe SDK — not JSON, not raw numbers.

- Closed-form DGP per measure
- Code-execution feedback loop
- 3-layer validator, no LLM re-call

3 AMORTIZED

Table Amortization

One Master Table → 10–30+ charts with arithmetic consistency guaranteed.

- 16 chart types × 7 relationships
- Deterministic SQL projection
- Cross-chart QA by construction

The 4-phase pipeline

LLM called only in phases 0 – 2. Phase 3 is fully deterministic.

PHASE 0

Domain Pool Construction

200+ fine-grained domains, cached

LLM · one-time

→

PHASE 1

Scenario Contextualization

Entities, metrics, temporal grain

LLM · per-sample

→

PHASE 2

Agentic Data Simulator (SDK)

Master Fact Table + Schema Metadata

LLM · per-sample

→

PHASE 3

View Amortization + QA Generation

10–30+ coherent multi-chart tasks

Deterministic

Code-execution feedback (max 3 retries)

OUTPUT

`{ Chart Images, Questions, Answers, Reasoning Chains } × N`

Phase 0 & 1: From domain pool to scenario

Chart types are never mentioned — data is born from business needs.

PHASE 0 · Domain Pool (one-time, cached)

15+ super-sectors

Healthcare | Finance | Retail | Transportation
Energy | Manufacturing | Education | Agriculture

213	15+	3	0.82
domains	topics	complexity tiers	diversity score

Complexity-balanced stratification

simple · 71	medium · 72	complex · 70

PHASE 1 · Scenario Contextualization

Sampled domain → concrete realistic scenario

```
{
  "scenario_title":
    "2024 H1 Shanghai Metro Ridership Log",
  "data_context":
    "Shanghai Transport Commission collected ... ",
  "temporal_granularity":
    "daily",
  "key_entities": [
     "Line 1", "Line 2", "Line 8", ...
  ],
  "key_metrics": [
     { "name": "daily_ridership",
       "unit": "10k passengers",
       "range": [5, 120] },
     { "name": "on_time_rate", ... }
  ],
  "target_rows": 900
}
```

Chart-type isolation is the key to breaking “template disease” in existing benchmarks.

Phase 2 paradigm shift: code-as-DGP

From brittle JSON configs to executable, type-safe Python.

BEFORE

LLM-as-Data-Generator

writes JSON configs or raw numbers

```
{ "columns": [
    { "name": "sales",
      "values": [42, 73, 61, 88]  },
    { "name": "profit_margin",
      "values": [0.12, 0.21, 0.18,
                 1.34]  ←
      }
  ], "correlation": 0.9 }
```

- ✗ **Numeric impossibilities slip through**
- ✗ **Correlations infeasible to enforce**
- ✗ **Only syntactic validation possible**

AFTER

LLM-as-Data-Programmer

writes Python calling a type-safe SDK

```
sim.add_category("hospital",
    values=["Xiehe","Huashan",...],
    group="entity")

sim.add_measure("wait_minutes",
    family="lognormal",
    param_model={...})

sim.declare_orthogonal(
    "entity", "patient")
```

- ✓ **Semantic errors caught at execution**
- ✓ **Typed exceptions → targeted LLM repair**
- ✓ **3-layer validator runs in milliseconds**

The FactTableSimulator SDK

Minimal, strongly-typed API — everything an LLM needs, nothing it doesn't.

SDK methods

STEP 1 · Column declarations

`add_category`	cat col • group • hierarchy via parent
`add_temporal`	time col • auto-derive DOW / month
`add_measure`	stochastic root • dist + effects
`add_measure_structural`	formula of other measures + noise

STEP 2 · Relationships & patterns

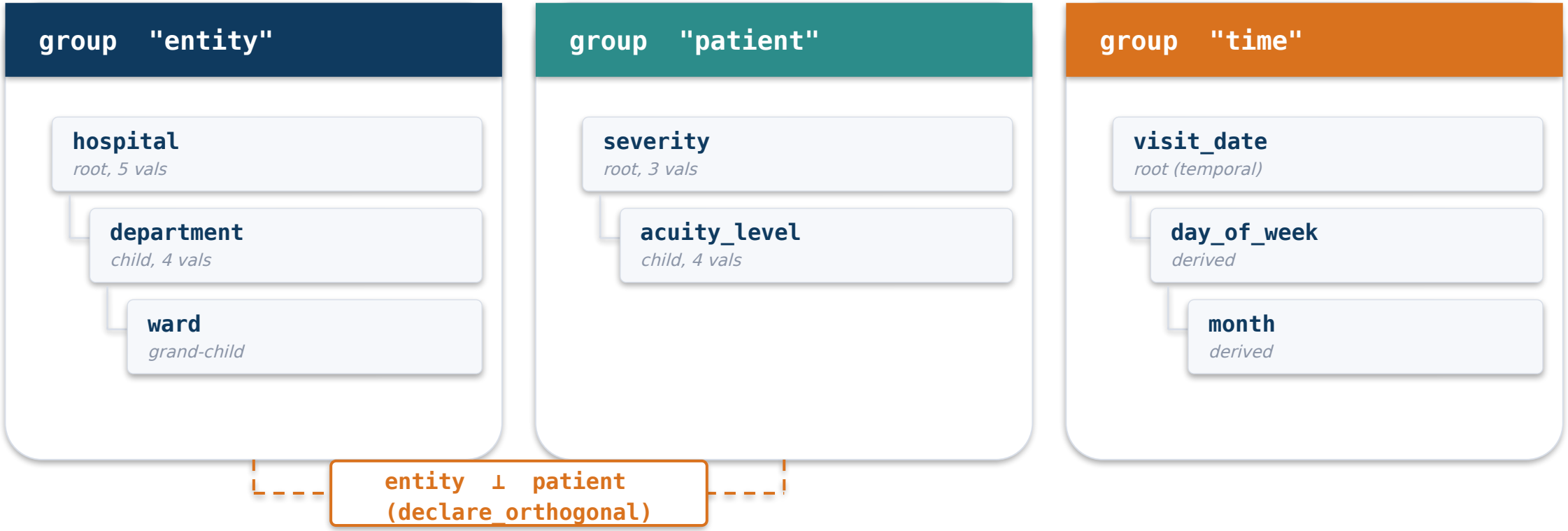
`declare_orthogonal`	group ⊥ group → propagates to pairs
`add_group_dependency`	root-level DAG across groups
`inject_pattern`	outlier / trend_break / reversal / ...
`set_realism`	missing / dirty / censoring (optional)

emergency_records.py

```
sim = FactTableSimulator(target_rows=500)

# Step 1 — declare columns
sim.add_category("hospital",
   values=["Xiehe","Huashan","Ruijin",...],
   group="entity")

sim.add_category("severity",
   values=["Mild","Moderate","Severe"],
   group="patient")

sim.add_measure("wait_minutes",
   family="lognormal",
   param_model={
     "mu": {"intercept": 2.8,
       "effects": {"severity": {...}}}})

sim.add_measure_structural("cost",
   formula="wait_minutes*12 + surcharge",
   noise={"family":"gaussian","sigma":30})

# Step 2 — relationships & patterns
sim.declare_orthogonal("entity","patient")
sim.inject_pattern("outlier_entity", ...)

df, meta = sim.generate()
```

Dimension groups & orthogonality

One abstraction unifies categorical, temporal, cross-group semantics.



Declare once at group level → propagates to all cross-group pairs → validated by χ^2 test

→ Enables Orthogonal-Slice dashboards & independence-based multi-chart QA

DAG-ordered event-level generation

Every row is one atomic event. No cross-product, no repetition.

LAYER 0
Independent roots

- **hospital** — *Cat*
- **severity** — *Cat*
- **visit_date** — *Temp*

LAYER 1
Dependent non-measures

- **department** — *Cat | hospital*
- **payment** — *Cat | severity*
- **day_of_week** — *derive(visit_date)*
- **month** — *derive(visit_date)*

LAYER 2
Stochastic measure

- **wait_minutes** — *LogNormal(μ(sev, hosp), σ(sev))*

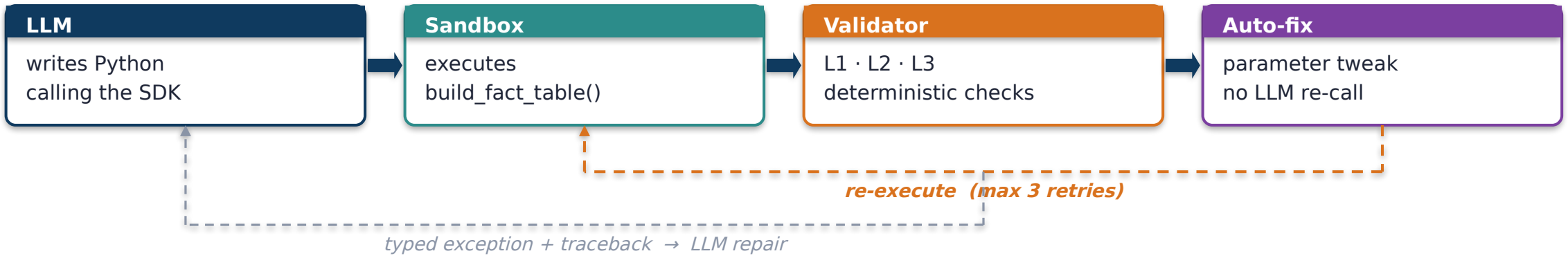
LAYER 3
Structural measures

- **cost** — *12·wait + surcharge(sev) + ε*
- **satisfaction** — *9 − 0.04·wait + adj(sev) + ε*

After DAG-ordered sampling: γ pattern injection → δ realism (optional) → τ post-process

Three-layer validation + code-execution feedback

Semantic errors, not just syntactic ones. Auto-fix without LLM re-call.



L1 Structural

- Row count within 10%
- Categorical cardinality
- Finite measures
- χ^2 for orthogonal pairs
- Measure DAG acyclicity

L2 Statistical

- KS test per predictor cell
- Structural residual mean
- Structural residual σ
- Conditional transitions
- Correlation ± 0.15

L3 Pattern

- Outlier $z \geq 2.0$
- Trend-break magnitude
- Ranking reversal sign
- Dominance-shift check
- Seasonal anomaly

Table amortization: one table → many tasks

Every derived chart shares the same ground-truth arithmetic by construction.

VIEW =

$$\sigma \circ \gamma \circ \pi \quad (M)$$

MASTER FACT TABLE

atomic events · 500–3000 rows

Comparison
bar · grouped_bar

Trend
line · area

Distribution
histogram · box · violin

Composition
pie · donut · stacked_bar · treemap

Relationship
scatter · bubble · heatmap · radar

Flow
waterfall · funnel

7 inter-chart relationships

- Drill-down
- Orthogonal Slice
- Comparative
- Dual-Metric
- Part-Whole
- Associative
- Causal Chain

10 – 30+
coherent tasks per table

1 LLM call → 1 script → 1 Master Table → 16 chart types × 7 relationships

Operator algebra: a question is a typed pipeline

Two types (V, S). Four operator families. Difficulty = # ops.

SET (V → V)

- Filter
- Sort
- Limit
- GroupBy

SCALAR (V → S)

- Max
- Min
- Avg
- Sum
- Count
- ArgMax
- ArgMin
- ValueAt

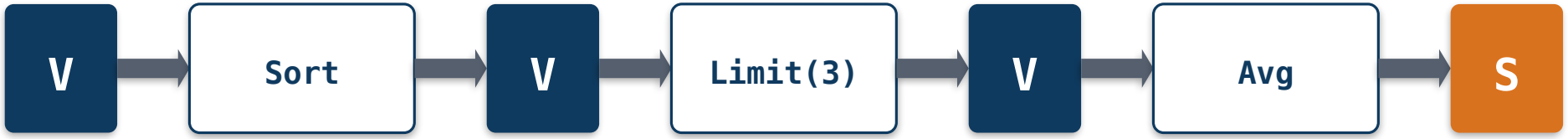
COMBINATOR (S,S → S)

- Diff
- Ratio

BRIDGE (S,V → V | V,V → S)

- EntityTransfer
- ValueTransfer
- TrendCompare
- RankCompare

Example pipeline (3 ops, MEDIUM)



QUESTION

"What is the average wait time of the top-3 hospitals?"

Cross-chart reasoning via bridge operators

A multi-chart question chains two pipelines through a typed bridge.

7 relationships → valid bridges

Drill-down	EntityTransfer · ValueTransfer
Orthogonal Slice	EntityTransfer · RankCompare
Comparative	ValueTransfer · TrendCompare
Dual-Metric	EntityTransfer · RankCompare · ValueTransfer
Part-Whole	EntityTransfer · ValueTransfer
Associative	RankCompare · EntityTransfer
Causal Chain	EntityTransfer · ValueTransfer

EXAMPLE · Dual-Metric with EntityTransfer

Chart A: hospital × wait

Chart B: hospital × cost

`V_a` → `ArgMax` → `'Xiehe'` → `Bridge` → `V_b` → `ValueAt` → `6200`

QUESTION · difficulty = 3 ops

"The hospital with the longest wait — what is its cost?"

Summary

One formal substrate — programmable, amortized, deterministic.

LAYER 1

Operator Algebra

Formalizes every chart QA as a typed pipeline.

MECHANISM

Set · Scalar · Combinator · Bridge (16 ops, 2 types)

LAYER 2

Agentic Simulator

LLM writes executable SDK code — not JSON, not raw numbers.

MECHANISM

Type-safe API · DAG measures · Orthogonality · Pattern injection

LAYER 3

Table Amortization

One Master Table serves 10–30+ coherent tasks.

MECHANISM

16 chart types × 7 inter-chart relationships

LAYER 4

Rule-Based QA

Deterministic, template-free generation with pattern-seeded hard QA.

MECHANISM

Intra-view · Inter-view · Pattern-triggered · Difficulty = # ops

CORE GUARANTEE

Every chart view shares the same ground-truth arithmetic by construction.